FedMATRIX-LoRA: Client-Aware Matrix-Level Adaptation for Resource-Constrained Federated Fine-Tuning

Anonymous Authors

***Abstract*—Federated fine-tuning must reconcile heterogeneous client data with limited training resources. For low-rank adaptation (LoRA), the allocation problem concerns not only adapter capacity but also which target matrices each client should update. This paper presents FedMATRIX-LoRA, a client-aware matrix-level adaptation framework for resource-constrained federated fine-tuning. Each client evaluates candidate matrices using a local mini-batch at the current global adapter and constructs feasible configurations from the resulting importance scores. The server coordinates client participation and matrix configurations under a shared training budget and a round-latency objective. Selected adapters are trained locally and aggregated matrix-wise. This design couples data-dependent adaptation preferences with resource feasibility without changing the LoRA parameterization. Experiments on image classification, language understanding, and dialogue summarization show improvements over the evaluated baselines in task metrics and in resource efficiency under the adopted analytical client profiles.**

***Index Terms*—Federated fine-tuning, low-rank adaptation, client-aware matrix selection, resource heterogeneity**

I. Introduction

Foundation models provide reusable representations, but their adaptation often depends on data held by different institutions or devices. Federated learning enables these data holders to optimize a shared model without pooling their raw examples [3], [4]. The scale of pretrained models, however, makes full-parameter updates expensive for clients with limited memory, computation, and communication capacity. Low-rank adaptation (LoRA) reduces the trainable parameter set by representing weight updates through compact factors while keeping the pretrained backbone fixed [5]. Federated LoRA thus makes collaborative adaptation more accessible, but it leaves an allocation question: which parts of the adapter should each client train with its available resources?

This question is not determined by adapter size alone. Training memory includes the frozen backbone, trainable parameters and their optimizer states, and intermediate activations required by backpropagation. Reducing rank shrinks the adapter-related state, but does not necessarily remove the backward computation needed to reach a trainable location. Fed-HeLLo identifies this distinction and reports limited total-memory savings from rank reduction when activations dominate [8]. Choosing trainable locations therefore complements rank adaptation: it changes both the parameters receiving updates and, under partial-adapter execution, the activation dependencies that must be retained. We study target selection at a fixed rank, separating this decision from the amount of capacity assigned within each selected adapter.

A Transformer block contains several target projections, such as Query, Key, Value, and feed-forward transformations. Each can receive a distinct LoRA update, so a block-level allocation couples choices that can instead be made separately. The LoRA placement study and AdaLoRA's adaptive rank allocation already establish that adaptation value varies across matrices and layers [5], [38]. These findings motivate treating the adapter subset as a decision variable rather than selecting a common location pattern and changing only its size. At a fixed rank, matrix-level selection allows a client to retain some projections within a block without assigning training capacity to every candidate projection in that block.

Selecting among these candidates requires an estimate of their value before committing a training budget. Importance scores provide such an estimate by measuring a parameter block's sensitivity to an adaptation objective. Gradient- and Fisher-based selection methods use information from the task loss to prioritize updates [8], [39]. The data used for this assessment are consequential in federated learning: two clients can start from the same global adapter yet optimize different local objectives because their sample distributions differ. Accordingly, matrix importance should be understood as conditional on the model state and the data distribution, rather than as a fixed attribute of the pretrained architecture. This raises the question of whether a single importance vector adequately represents the adaptation priorities of heterogeneous clients.

Fed-HeLLo estimates layer importance on server-held proxy data and uses the resulting scores to guide client allocations [8]. A proxy distribution provides a shared reference, but may assign priorities different from those induced by an individual client's objective. Fed-pilot further incorporates local and globally aggregated importance information into memory-constrained LoRA allocation [40]. Building on importance-guided adaptation, we investigate individual target-projection choices within each client and coordinate those choices under local memory and latency limits and a shared round budget. The resulting problem couples the source of importance information, the granularity of selection, and the feasibility of executing the selected updates.

A controlled motivating comparison is designed to distinguish differences in matrix priorities from their consequences for local adaptation. On CIFAR100 with ViT-base, it contrasts

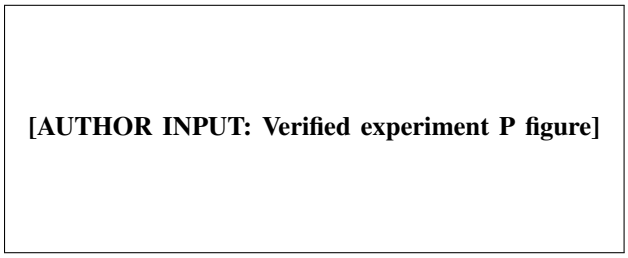


Fig. 1. Planned motivating comparison of scoring sources under data heterogeneity. (a) Jaccard distance between Local and Server-Proxy matrix selections. (b) Held-out local validation-loss reduction for Local, Local-Mean, and Server-Proxy at matched matrix counts, ranks, and training lengths. **[AUTHOR INPUT: Insert the verified P panels, seed set, aggregation unit, and uncertainty definition.]**

Local, which assigns each client its own scores; Local-Mean, which averages client scores into one shared vector; and Server-Proxy, which evaluates a separate server-held proxy set. The comparison spans IID data and Dirichlet partitions with concentration parameters 1.0 and 0.1. It holds the score formula, starting adapter, selected-matrix count, rank, local training length, and feasibility checks fixed within each comparison. Selection disagreement is measured separately from the reduction in held-out local validation loss. Local-Mean isolates the effect of sharing the same locally computed evidence, whereas Server-Proxy examines a distinct scoring-data source. The two planned panels in Fig. 1 connect these questions without equating different selections with better selections. **[AUTHOR INPUT: Experiment P results are pending. Insert the verified selection-disagreement and paired validation-loss findings, with uncertainty, before drawing an empirical conclusion.]**

The methodological challenge has two parts. First, importance assessment must distinguish client-specific adaptation priorities at the current global model without requiring trial training of every candidate configuration. A low-cost gradient-based profile addresses this assessment problem. Second, those priorities must be translated into feasible allocations. A high-scoring subset can exceed a client's memory limit, and configurations that are feasible individually can jointly exceed the round budget or incur excessive synchronization latency. Thus, neither a shared importance ranking nor independent local selection alone specifies the desired training round. Client-conditioned assessment and resource-feasible coordination must be connected.

We propose FedMATRIX-LoRA, a client-aware matrix-level adaptation framework for resource-constrained federated fine-tuning. During profiling refreshes, each client evaluates candidate LoRA factors on a local mini-batch at the current global adapter. The resulting scores define configuration priorities, while the resource model determines which configurations are feasible. Clients generate candidate configurations at different matrix-equivalent costs; the server coordinates participation and configuration choice under the shared budget and latency objective. Selected adapters are trained locally and aggregated matrix-wise to update a shared global adapter. In this design, local evidence determines preferred adaptation locations, and resource coordination determines which proposed updates can be executed together.

Our contributions are threefold:

- We formulate client-aware target-matrix selection for resource-constrained federated fine-tuning, making the trainable projection subset responsive to local data and device feasibility rather than specifying it solely through adapter capacity.
- We develop FedMATRIX-LoRA, which connects local gradient-based profiling with feasible configuration generation and server coordination. The framework supports distinct projection choices within a Transformer block and across clients while maintaining a shared global adapter through matrix-wise aggregation.
- We provide a conditional stationarity analysis that relates selective aggregation to effective-update bias and matrix coverage, and evaluate the complete framework on image classification, language understanding, and dialogue summarization under heterogeneous data and analytical resource profiles.

Section II positions the work relative to prior studies. Section III presents the system model and problem formulation, and Section IV describes FedMATRIX-LoRA and its analysis. Section V reports the evaluation, followed by the conclusion in Section VI.

II. Related Work

Parameter-efficient and selective adaptation. Parameter-efficient fine-tuning changes either the representation of an update or the subset of parameters that is trained. LoRA represents updates through low-rank factors attached to pretrained matrices, while QLoRA combines a quantized frozen backbone with trainable low-rank adapters [5], [10]. These mechanisms reduce different components of the adaptation footprint. Target selection introduces a further choice: an available adapter need not be trained on every client in every round. The distinction is useful in heterogeneous systems, where a common candidate architecture can admit different feasible trainable subsets.

Importance-aware methods provide a basis for making such choices. AdaLoRA assigns nonuniform rank budgets across weight matrices through importance-guided pruning of an SVD-style update parameterization [38]. FISH constructs a sparse trainable-parameter mask from Fisher-based importance scores [39]. These methods demonstrate that uniform allocation is not the only way to organize a limited adaptation budget. FedMATRIX-LoRA applies importance-guided selection to a different decision: the fixed-rank target subset assigned to each federated client. Whereas within-matrix rank allocation controls adapter capacity, client-conditioned target selection determines the projections receiving that client's updates. Resource coordination then determines which client configurations can be executed together.

Heterogeneous federated LoRA. HetLoRA supports heterogeneous client ranks and combines local rank self-pruning with server-side sparsity-weighted aggregation [6]. FAH-QLoRA

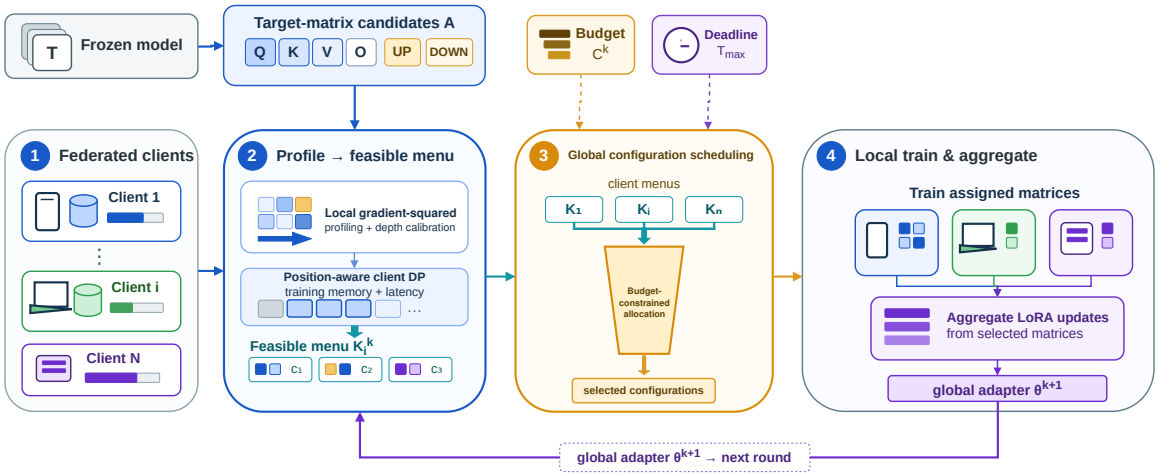


Fig. 2. FedMATRIX-LoRA workflow. Client-conditioned profiling scores target matrices; position-dependent training-memory and client-latency constraints shape feasible configurations; configuration-level global coordination selects among them under the round budget.

jointly considers quantization and LoRA rank in federated resource adaptation [7]. These approaches establish the value of adapting capacity or numerical precision to client resources. As discussed by Fed-HeLLo, however, reducing rank primarily decreases optimizer-state memory in activation-dominated settings, motivating partial-layer training as a complementary allocation dimension [8]. LEGEND jointly configures tuning depth and rank distribution, explicitly connecting the location of trainable layers with resource use [32]. Its combination of depth and rank illustrates that placement and capacity can jointly affect the resource–performance trade-off.

Fed-HeLLo further accounts for unequal layer importance through scores computed on server-held proxy data and geometric allocation strategies [8]. Its score-guided strategy converts a shared assessment into allocation probabilities; different clients can receive different masks even though the underlying importance evidence is shared. FedMATRIX-LoRA instead estimates importance on each client's data and exposes separate projection choices inside a block. This separates two aspects of heterogeneity: differences in client training capacity and differences in the data-dependent priorities used to allocate that capacity.

Fed-pilot is the closest importance-guided allocation counterpart. In its block-indexed formulation, local and cross-client, temporally aggregated information-gain scores guide memory-constrained LoRA allocation, and compensated aggregation addresses uneven updates [40]. FedMATRIX-LoRA develops a different allocation formulation: independent target-projection indicators within blocks are coordinated across clients under memory limits, latency constraints, and a shared round budget. Local profiling is performed over the candidate target set at refresh rounds, and the resulting configurations are compared at the server. The formulation thereby connects within-block projection choices and client-specific importance with the feasibility of a jointly scheduled training round.

Aggregation and resource coordination. Aggregation and allocation address related but distinct parts of federated adaptation. FedEx-LoRA treats the mismatch introduced by aggregating low-rank factors, while LoRA-FAIR studies aggregation and initialization refinement [16], [17]. These directions concern how local updates are combined. Resource-aware methods such as Oort, FedBalancer, and PyramidFL address participant selection or training-work coordination under heterogeneous conditions [26]–[28]. In FedMATRIX-LoRA, the server decision also includes the trainable target subset assigned to a participating client. Feasible configurations therefore connect local adaptation preferences to the round-level allocation problem, rather than treating client participation and the internal trainable subset as unrelated decisions. The matrix-wise analysis characterizes the update pattern induced by this selective execution.

III. System Model and Problem Formulation

A. System Model and Federated LoRA Preliminaries

We consider a heterogeneous federated LoRA fine-tuning system consisting of one server and N clients. Each client i owns a private local dataset $\mathcal{D}_i$, an on-device training-memory capacity $M_i^{\max}$ (corresponding to the available GPU device memory in our implementation), an estimated communication bandwidth B_i, and heterogeneous computational resources. To preserve data privacy, all local datasets remain on their corresponding clients throughout the training process and are never uploaded to the server. The server initializes a pretrained backbone model Θ^0, which remains frozen during federated fine-tuning, and maintains the global LoRA adapter parameters θ^k at communication round k.

Let $n_i = |\mathcal{D}_i|$ denote the number of local training samples at client i. Federated LoRA fine-tuning aims to minimize the following global objective

$$F(\theta) = \frac{1}{N} \sum_{i=1}^{N} F_i(\theta), \tag{1}$$

where

$$F_i(\theta) = \frac{1}{n_i} \sum_{z \in \mathcal{D}_i} \ell(\Theta^0, \theta; z), \tag{2}$$

and $\ell(\cdot)$ denotes the task loss evaluated using the frozen backbone Θ^0 and trainable LoRA parameters θ.

For a target linear transformation $W_j \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA parameterizes its update as

$$\Delta W_j = B_j A_j, \tag{3}$$

where $A_j \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B_j \in \mathbb{R}^{d_{\text{out}} \times r}$ are low-rank matrices with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$.

We treat the trainable adapter subset as a client-specific decision. A target matrix denotes a pretrained projection, such as Query, Key, or Value; selecting it means updating its associated LoRA factors, not the frozen pretrained matrix itself. The rank of every selected adapter is fixed to the same value, so configuration differences arise from target selection.

B. Client-Aware Matrix-Level LoRA Allocation

We formulate federated LoRA allocation at the target-matrix granularity. Let

$$\mathcal{A} = \{W_1, W_2, \ldots, W_L\} \tag{4}$$

denote the ordered set of LoRA-compatible target matrices extracted from the backbone. These matrices include the Query, Key, Value, output, and feed-forward projection matrices in Transformer blocks.

For client i at communication round k, the matrix activation decision is represented by the binary vector

$$\mathbf{m}_i^k = [m_{i,1}^k, \cdots, m_{i,L}^k], \tag{5}$$

where

$$m_{i,j}^k = \begin{cases} 1, \text{if } W_j \text{ is activated for local training;} \\ 0, \text{otherwise.} \end{cases} \tag{6}$$

Each activation vector therefore specifies the subset of LoRA target matrices trained by client i during communication round k.

Let $V_i^k(\mathbf{m}_i^k)$ denote the profiled utility of client i's matrix configuration. It is computed from local data at the current global adapter and summarizes the estimated priority of the selected updates. We set $V_i^k(\mathbf{0}) = 0$. The profiling procedure is defined in Section IV-B.

Layer-group allocation is recovered by constraining all candidate matrices in a Transformer block to share the same selection indicator.

C. Resource Constraints and Problem Formulation

The activation decision of LoRA target matrices determines the resource consumption of each participating client. We model the training cost, training-memory usage, and training latency separately.

First, let $c_i(\mathbf{m}_i^k)$ denote the normalized training cost associated with matrix configuration $\mathbf{m}_i^k$. We approximate this cost by the number of activated rank-8 matrices, $c_i(\mathbf{m}_i^k) = \sum_{j=1}^{L} m_{i,j}^k$. This standardized count is not a measurement of matrix-specific computation or memory, which can differ even at the same rank; memory is modeled separately below.

Training-memory consumption depends on both the number and positions of activated matrices. For any nonzero $\mathbf{m}$, let $\mathcal{J}(\mathbf{m}) = \{j \mid m_j = 1\}$ and $j_{\min}(\mathbf{m}) = \min \mathcal{J}(\mathbf{m})$. Following the training-memory decomposition commonly adopted in partial-layer federated LoRA fine-tuning [8], we set $M_i(\mathbf{0}) = M_i^{\text{fixed}}$, while a nonzero configuration uses

$$M_i(\mathbf{m}) = M_i^{\text{fixed}} + q_i(r) c_i(\mathbf{m}) + \sum_{j=j_{\min}(\mathbf{m})}^{L} \mu_{i,j}, \tag{7}$$

where M_i^{fixed} denotes the memory occupied by the frozen backbone and fixed runtime buffers, $q_i(r)$ is the parameter-and-optimizer memory per activated rank-r LoRA matrix, and $\mu_{i,j}$ is retained activation memory. The last term reflects the position-dependent memory characteristic of Transformer fine-tuning: activating an earlier matrix requires retaining a longer backward propagation path and therefore consumes more activation memory.

The latency of client i consists of local computation and communication. Let γ_i denote its computation coefficient, σ_i the communication size per activated matrix, and B_i its available bandwidth. Its latency is

$$\begin{aligned} T_i^{\text{comp}}(\mathbf{m}_i) &= \gamma_i c_i(\mathbf{m}_i), \\ T_i^{\text{comm}}(\mathbf{m}_i) &= \frac{\sigma_i c_i(\mathbf{m}_i)}{B_i}, \\ T_i(\mathbf{m}_i) &= T_i^{\text{comp}}(\mathbf{m}_i) + T_i^{\text{comm}}(\mathbf{m}_i). \end{aligned} \tag{8}$$

The coefficients define an analytical resource profile; equal matrix-equivalent costs need not correspond to identical measured hardware costs.

At communication round k, the server jointly determines the participating clients and their matrix configurations under a shared resource budget. Let S_k denote the set of candidate clients, $z_i^k \in \{0, 1\}$ indicate whether client i participates in the current round, C^k denote the round-level training budget, and τ^k represent the latency of the current communication round. Let $\ell_i^{\min}$ and $\ell_i^{\max}$ bound the number of matrices activated by participating client i, and let $\lambda > 0$ control the utility–latency trade-off.

$$\max_{\{\mathbf{m}_i^k, z_i^k\}, \tau^k} \quad \sum_{i \in S_k} V_i^k(\mathbf{m}_i^k) - \lambda \tau^k \tag{9}$$

$$\text{s.t.} \quad \sum_{i \in S_k} c_i(\mathbf{m}_i^k) \le C^k, \tag{10}$$

$$m_{i,j}^k \le z_i^k, \tag{11}$$

$$\ell_i^{\min} z_i^k \le \sum_{j=1}^{L} m_{i,j}^k \le \ell_i^{\max} z_i^k, \tag{12}$$

$$M_i(\mathbf{m}_i^k) \le M_i^{\max}, \quad \forall i : z_i^k = 1, \tag{13}$$

$$T_i(\mathbf{m}_i^k) \le \tau^k \le T_{\max}, \quad \forall i : z_i^k = 1. \tag{14}$$

The objective maximizes the aggregated client-conditioned adaptation utility while minimizing round latency. Equations (10)–(14) impose the round-level matrix-equivalent budget, participation consistency, activated-matrix bounds, per-client training-memory limits, and the round deadline. In particular, Eqs. (11) and (12) together ensure that an inactive client selects no matrix, whereas an active client selects between $\ell_i^{\min}$ and $\ell_i^{\max}$ matrices.

The formulation couples client-level feasibility with a shared round budget. We implement it by constructing feasible configurations locally and coordinating these configurations at the server.

IV. FedMATRIX-LoRA Design

A. Framework Overview

FedMATRIX-LoRA links data-dependent matrix assessment to resource-feasible execution. As illustrated in Fig. 2, clients profile candidate matrices at the current global adapter and construct configurations satisfying their local limits. The server coordinates these configurations under the round budget and latency objective. Clients then update the assigned adapters, and their results are aggregated matrix-wise. The scoring mechanism and the allocation procedure are described separately below.

B. Client-Conditioned Matrix Profiling

Client-conditioned assessment. Profiling supplies an importance estimate before a client's training configuration is selected. At a common global adapter, the local loss depends on the client's examples; its gradient therefore describes sensitivity to the parameters under that client's adaptation objective. A shared score vector summarizes a common reference objective, whereas separate local evaluations preserve differences among client objectives. We use this distinction to assign update priorities at the target-matrix level. The goal is to inform which LoRA factors a client should train, while the resource constraints in Section IV-C determine which combinations can be executed.

A single-pass gradient score. Gradient- and Fisher-based criteria have been used to select trainable parameters and LoRA layers [8], [39]. We adopt a gradient-squared criterion because it can be evaluated for all candidate targets in one backward pass, without separately training each proposed configuration. During a refresh at round k, client i attaches LoRA modules to the candidate projections, loads the current global adapter θ^k, and evaluates the average loss on a local mini-batch $\mathcal{B}_i^k$. Let θ_j^k denote the LoRA factors associated with target matrix j. Its raw score is

$$g_{i,j}^k = \sum_{p \in \theta_j^k} \left(\nabla_p \mathcal{L}_i(\theta^k; \mathcal{B}_i^k) \right)^2 . \tag{15}$$

The score measures the aggregate magnitude of the local batch-loss gradient over the adapter's trainable coordinates. Squaring makes contributions from different parameter coordinates nonnegative, and summation gives a block-level measure rather than an average sensitivity per parameter. For a fixed parameterization, a larger gradient norm indicates greater immediate loss sensitivity to a perturbation of that block, motivating its use as an update-priority signal. Because the calculation uses the gradient of a batch-averaged loss, we describe it as a Fisher-inspired proxy rather than an exact Fisher estimate. Its role is to rank candidate updates before local training, not to substitute for the observed task metric after training.

Both the local examples and θ^k enter the score. Consequently, clients may assign different priorities to the same target even when their models begin a round from the same global state. Re-evaluation at a later refresh also allows the priorities to change as the global adapter evolves. The single-pass design obtains a score for every candidate at a refresh, including candidates not selected for that client's preceding local update. This avoids restricting the next allocation decision to the client's previously active targets, without requiring trial optimization of the candidate configurations.

Shared positional calibration. The data-dependent score is supplemented by a structural preference over candidate positions. Prior work on partial LoRA fine-tuning reports that deeper adaptation can provide favorable performance–resource trade-offs in the studied settings [32]. We incorporate this observation through a multiplicative depth factor:

$$\widetilde{g}_{i,j}^k = g_{i,j}^k (1 + d_j)^\rho, \tag{16}$$

Here $d_j = (j-1)/(L-1)$ is the normalized position in the ordered candidate set. For the evaluated configurations, $\rho = 1 + 1.5\mathbb{I}_{\text{pair}} + \eta$, where $\mathbb{I}_{\text{pair}}$ indicates a sentence-pair task and $\eta = 0, 1, 2$ denotes IID, ordinary non-IID, and severe non-IID settings, respectively. This is the specified calibration schedule; the local dependence remains in $g_{i,j}^k$.

The multiplier applies a common positional preference without replacing the client-specific evidence. It can change an individual client's ranking, but does not force clients with different raw scores to share a ranking or select the same matrices. This separates the source of importance information from the structural prior applied to it. The No-Depth comparison in Section V-D is designed to isolate the contribution of this calibration while retaining the local scoring and allocation procedures.

Configuration utility and refresh. The server coordinates configurations proposed by different clients. We therefore use a common normalization scale rather than assigning a separate unit scale to each client's largest score:

$$G_{\max}^k = \max_{\substack{h \in S_k \\ 1 \le j \le L}} \widetilde{g}_{h,j}^k, \qquad v_{i,j}^k = \frac{\widetilde{g}_{i,j}^k}{\max\{G_{\max}^k, \delta\}} \in [0,1], \tag{17}$$

The positive constant δ prevents division by zero. Using the same divisor preserves the relative magnitudes of the calibrated scores across both clients and matrices, and sets the utility scale used in the server's utility–latency trade-off. For assignment $\mathbf{m}_i^k$, the configuration utility is $V_i^k(\mathbf{m}_i^k) = \sum_{j=1}^L m_{i,j}^k v_{i,j}^k$. Thus, local data determine the raw priorities, the depth factor

supplies a structural preference, and the common scale makes those scores directly usable in the shared allocation objective. The sum is a configuration-selection proxy; it does not assume that the full training loss separates into independent matrix losses.

Scores are cached between designated refresh rounds and recomputed when the cache is unavailable or a refresh is due. Reuse avoids repeating the profiling pass at every allocation decision, while a refresh reconnects the score vector to the current adapter. Raw examples and profiling gradients remain local; coordination uses matrix scores and configuration summaries. The profiling sample count and refresh schedule are therefore part of the scoring protocol, rather than additional optimization variables. Section V-C specifies the controlled test of whether configuration scores rank short-term held-out loss reductions, and Section V-D specifies the end-to-end comparisons of scoring source, selection granularity, and calibration.

C. Resource-Feasible Matrix Allocation

For a fixed round k, superscripts are omitted from configuration attributes and scheduling variables when unambiguous. Each feasible configuration κ generated by client i is represented as

$$\kappa = (V_{i\kappa}, c_{i\kappa}, T_{i\kappa}, M_{i\kappa}, \mathbf{m}_{i\kappa}),$$

where $V_{i\kappa}$, $c_{i\kappa}$, $T_{i\kappa}$, $M_{i\kappa}$, and $\mathbf{m}_{i\kappa}$ denote the profiled adaptation utility, normalized matrix-equivalent cost, estimated latency, estimated training-memory consumption, and binary target-matrix assignment, respectively. Each client uploads a compact configuration menu $\mathcal{K}_i^k$.

Local configurations. Under the adopted memory model, feasibility depends on the selected-matrix count and the earliest selected position through the retained-activation term. For each count c, the client therefore searches over feasible earliest positions and retains a configuration with the largest profiled utility. Let $Q_{i,e}^k(t)$ denote the maximum utility of selecting t matrices from the suffix $\{e, \dots, L\}$. Its recursion and the utility for configurations whose earliest selected position is e are: $Q_{i,e}^k(0) = 0$ and $Q_{i,L+1}^k(t > 0) = -\infty$.

$$\begin{aligned} Q_{i,e}^k(t) &= \max\left\{Q_{i,e+1}^k(t), v_{i,e}^k + Q_{i,e+1}^k(t-1)\right\}, \quad t \geq 1, \\ \Phi_{i,e}^k(c) &= v_{i,e}^k + Q_{i,e+1}^k(c-1). \end{aligned} \tag{18}$$

Only configurations satisfying the matrix-count, memory, and latency constraints are retained. Under the current count-based latency model, configurations with the same cost consume the same server budget and have the same latency; retaining the highest-utility feasible configuration at each cost therefore suffices for subsequent coordination.

Server coordination. The server enumerates the distinct feasible latency thresholds $\tau \leq T_{\max}$. For each threshold, it removes configurations exceeding τ and solves the remaining multiple-choice knapsack problem, choosing at most one configuration per client. With $H_i(w)$ denoting the largest utility attainable from the first i client menus at budget w, the recursion is:

$$H_i(w) = \max_{\kappa \in \mathcal{K}_i^k(\tau) \cup \{\varnothing\} : c_{i\kappa} \leq w} \{H_{i-1}(w - c_{i\kappa}) + V_{i\kappa}\}, \tag{19}$$

Here $\mathcal{K}_i^k(\tau)$ is the menu after latency filtering, $\varnothing$ is the zero-utility inactive choice with zero cost, and $H_0(w) = 0$.

The server selects the threshold and configuration assignment with the largest total utility minus $\lambda\tau$. This procedure optimizes the stated profiled objective; its role is to realize client-conditioned selection under the resource model. Algorithm 1 summarizes the complete training round.

D. Local Training and Matrix-Wise Aggregation

After determining the optimal configuration assignment, the server dispatches the global adapter θ^k together with the selected matrix assignments to corresponding clients. Only selected LoRA parameters are updated locally. Task-specific trainable modules, if required by the downstream task, are optimized jointly with the selected LoRA parameters.

Since different clients may update different subsets of LoRA matrices, the aggregation process is performed independently for each trainable matrix. Let $\mathcal{U}_j^k$ denote the clients returning updates for matrix j, and let $\theta_{i,j}^{k+1}$ be the corresponding locally updated LoRA parameters from client i. The server performs matrix-wise client-uniform aggregation:

$$\theta_j^{k+1} = \begin{cases} |\mathcal{U}_j^k|^{-1} \displaystyle\sum_{i \in \mathcal{U}_j^k} \theta_{i,j}^{k+1}, & |\mathcal{U}_j^k| > 0, \\ \theta_j^k, & |\mathcal{U}_j^k| = 0. \end{cases} \tag{20}$$

The updated global adapter is used to initialize the next round. Because the updating client set can differ across target matrices, the aggregation averages only the clients contributing to each matrix and preserves the previous value when no update is available.

E. Convergence of Matrix-Wise Aggregation

We analyze the effective update induced by the matrix-wise aggregation rule. For this analysis, θ includes the LoRA parameters and any jointly trained shared task modules. Let P^k project onto the parameter blocks selected for update in round k, as determined before local training. For a common positive round scale γ, define the actual aggregated direction by $v^k = (\theta^k - \theta^{k+1})/\gamma$. Conditioning on the current model, profiling information, and selected configuration, denoted by $\mathcal{H}_k$, write

$$b^k = \mathbb{E}[v^k \mid \mathcal{H}_k] - P^k \nabla F(\theta^k), \tag{21}$$

$$\xi^k = v^k - \mathbb{E}[v^k \mid \mathcal{H}_k]. \tag{22}$$

Thus $\mathbb{E}[\xi^k \mid \mathcal{H}_k] = 0$, and the update is

$$\theta^{k+1} = \theta^k - \gamma\big(P^k \nabla F(\theta^k) + b^k + \xi^k\big). \tag{23}$$

The bias b^k accounts for selective aggregation and the difference between local optimizer updates and the selected global gradient. This representation does not replace the experimental AdamW optimizer with SGD.

Algorithm 1 FedMATRIX-LoRA in communication round k

Require: Global adapter θ^k, candidate matrix set $\mathcal{A}$, candidate clients S_k, budget constraint C^k, latency bound $T_{\max}$

Client-conditioned matrix profiling and allocation:

for all $i \in S_k$ **do**
  if profiling cache is unavailable or refresh is required **then**
    Insert LoRA modules on candidate matrices $\mathcal{A}$
    Compute matrix importance scores $g_{i,j}^k$ using one local mini-batch
    Apply position-aware calibration to obtain $\widetilde{g}_{i,j}^k$
  else
    Reuse cached matrix importance scores
  end if
  Normalize matrix utilities to obtain $v_{i,j}^k$
  Generate feasible configuration menu $\mathcal{K}_i^k$ via suffix dynamic programming
end for

Resource-feasible server coordination:

Initialize best objective value $U^* \leftarrow -\infty$
for all distinct feasible latency thresholds $\tau \le T_{\max}$ **do**
  Remove configurations satisfying $T_{i\kappa} > \tau$
  Solve MCKP using dynamic programming under budget constraint C^k
  Compute objective value $\sum_i \sum_\kappa x_{i\kappa} V_{i\kappa} - \lambda\tau$
  if current objective $> U^*$ **then**
    Update the best configuration assignment
  end if
end for

Local training and matrix-wise aggregation:

Dispatch selected matrix assignments and θ^k to clients
for all selected clients **do**
  Train LoRA parameters only on assigned matrices
end for
Perform matrix-wise aggregation to obtain θ^{k+1}
return updated global adapter θ^{k+1}

Assumptions. Assume that F is β-smooth on the relevant update segments and bounded below by $F_{\inf}$. Suppose the effective update errors satisfy

$$\mathbb{E}\|b^k\|^2 \le B_{\mathrm{upd}}^2, \qquad \mathbb{E}\|\xi^k\|^2 \le S_{\mathrm{upd}}^2. \tag{24}$$

We additionally assume effective matrix coverage: for some $0 < \alpha_{\mathrm{cov}} \le 1$ and $\varepsilon_{\mathrm{cov},k} \ge 0$,

$$\mathbb{E}\|P^k \nabla F(\theta^k)\|^2 \ge \alpha_{\mathrm{cov}} \mathbb{E}\|\nabla F(\theta^k)\|^2 - \varepsilon_{\mathrm{cov},k}. \tag{25}$$

This condition quantifies the global gradient energy captured by the selected blocks.

Theorem 1 (Expected stationarity)**.** *Under the stated assumptions, for $0 < \gamma \le 1/\beta$ and K rounds,*

$$\frac{1}{K}\sum_{k=0}^{K-1} \mathbb{E}\|\nabla F(\theta^k)\|^2 \le \frac{2D_0}{\alpha_{\mathrm{cov}}\gamma K} + \frac{B_{\mathrm{upd}}^2 + \beta\gamma S_{\mathrm{upd}}^2 + \bar{\varepsilon}_{\mathrm{cov}}}{\alpha_{\mathrm{cov}}}, \tag{26}$$

where $D_0 = F(\theta^0) - F_{\inf}$ for a fixed initialization and $\bar{\varepsilon}_{\mathrm{cov}} = K^{-1}\sum_k \varepsilon_{\mathrm{cov},k}$.

For a fixed horizon, setting $\gamma = c/\sqrt{K} \le 1/\beta$ yields an $O(K^{-1/2})$ optimization and noise term when the error bounds and coverage constant are uniform across the analyzed runs. The residual is governed by effective-update bias and uncovered gradient energy. The result characterizes average stationarity, rather than optimal test accuracy.

The contribution of client selection can be made explicit. Let $n_j = |\mathcal{U}_j| > 0$, $g_{i,j} = \nabla_j F_i(\theta)$, $g_j = N^{-1}\sum_i g_{i,j}$, and $\zeta_j^2 = N^{-1}\sum_i \|g_{i,j} - g_j\|^2$.

Lemma 1 (Matrix-wise selection bias)**.** *For any updating-client set $\mathcal{U}_j$,*

$$\left\| \frac{1}{n_j}\sum_{i\in\mathcal{U}_j} g_{i,j} - g_j \right\|^2 \le \left(\frac{N}{n_j} - 1\right)\zeta_j^2. \tag{27}$$

The bound relates aggregation bias to participation and gradient heterogeneity at each target matrix. Proofs are provided in Appendix A.

V. Experiments

In this section, we evaluate the effectiveness of the proposed FedMATRIX-LoRA through extensive simulations across diverse datasets and model settings.

A. Evaluation Setup

We evaluate image classification on CIFAR100 [33], language understanding on MNLI, QNLI, QQP, and SST-2 from GLUE [34], and dialogue summarization on SAMSum [35].

Table I gives the backbones, partitions, rounds, and rank-8 matrix-cost budgets; each round has ten clients. Classification uses equal random IID partitions or Dirichlet non-IID partitions with $\alpha = 1.0$ or 0.1. SAMSum uses contiguous length-sorted shards for severe skew and randomly assigned length-sorted shards for mild skew.

Federated runs use NVIDIA A800-SXM4-80GB GPUs. Table II gives the analytical client resource profiles and training settings. The main comparisons use seed 2024 with shared partitions and resource draws. The additional scoring and ablation studies report their seed sets and data splits separately. Cumulative training cost $\overline{C}^K = \sum_{k=1}^K \sum_i c_i(\mathbf{m}_i^k)$ excludes profiling. Elapsed-time curves accumulate the analytical latency in Eq. (8), not end-to-end wall-clock time.

We report final and best Accuracy for classification and ROUGE-L for SAMSum, without averaging the two metrics.

We compare FedMATRIX-LoRA with representative baselines, including homogeneous LoRA training (FedAvg-LoRA) [3], [5], uniform matrix allocation (Uniform), layer-wise allocation (Fed-HeLLo) [8], layer-group heuristic allocation (MS), heterogeneous-rank allocation (LEGEND [32] and HETLoRA-style [6]), and joint quantization/rank allocation (FAH-QLoRA-style) [7]. These baselines represent different adaptation granularities and resource-allocation strategies, enabling a comprehensive comparison with our client-aware matrix scheduling. For a fair comparison, all methods use the same backbone models, data partitions, communication rounds, local training settings, and evaluation protocol.

TABLE I

EVALUATION SETTINGS. NON-IID x: DIRICHLET $\alpha = x$; LEN-M/S: MILD/SEVERE LENGTH SKEW; SLASH-SEPARATED BUDGETS FOLLOW PARTITION ORDER.

Dataset	Task	Backbone	Partitions	Rounds	Cum. budget	Metric
CIFAR100	Image classification	ViT-base [2]	IID, Non-IID 1.0/0.1	180	7,200/12,000	Accuracy
MNLI	Language inference	DeBERTa-v3-large [36]	IID, Non-IID 1.0	50	9,500	Accuracy
QNLI	Sentence-pair classification	RoBERTa-base [37]	IID, Non-IID 1.0	80	4,800/7,200	Accuracy
QQP	Paraphrase classification	DeBERTa-v3-large [36]	IID, Non-IID 1.0	50	9,500	Accuracy
SST-2	Sentiment classification	RoBERTa-base [37]	IID, Non-IID 1.0	80	7,200	Accuracy
SAMSum	Dialogue summarization	OPT-1.3B [1]	IID, Len-M, Len-S	40	10,000	Rouge-L

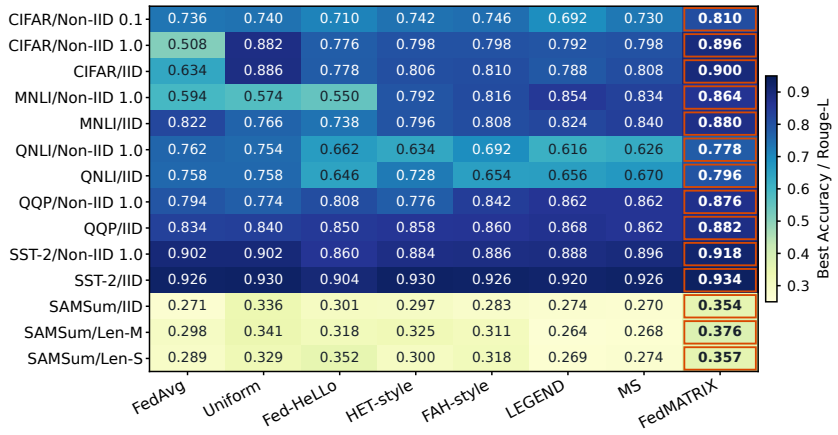


Fig. 3. Best task metric across 14 settings. Boxes mark row maxima; SAMSum uses Rouge-L and all other rows Accuracy.

TABLE II

SYSTEM AND TRAINING PROFILE.

Control	Reported configuration
Clients	10
Training-memory budget	40/25/15 GB; ratio 6:3:1
Compute factor	0.8–2.0 s per rank-8 matrix-equivalent
Bandwidth	5–50 MB/s
LoRA rank / profiling	8 / one local mini-batch
Local training cap	one epoch, at most 50 mini-batch steps
Optimizer	AdamW
Batch size / learning rate	1–32 / 10^{-5}–10^{-4}
Default latency cap	200 s per round

B. Task Performance and Resource Efficiency

Fig. 3 compares the best observed task metric across 14 evaluation settings. As shown in Fig. 3, FedMATRIX-LoRA consistently ranks first across all CIFAR100, GLUE, and SAMSum settings, covering both IID and heterogeneous data partitions. Compared with heterogeneous-rank and quantization-based baselines, FedMATRIX-LoRA also delivers substantially higher peak performance. For example, on MNLI IID, HETLoRA-style and FAH-QLoRA-style achieve 0.796 and 0.808, respectively, whereas FedMATRIX-LoRA reaches 0.880. On CIFAR100 IID, the corresponding accuracies are 0.806, 0.810, and 0.900, showing the performance of the complete framework under the reported evaluation protocol.

The budget–quality trajectories in Fig. 4 characterize the performance obtained as the cumulative matrix-equivalent cost increases. Together with the matched-target comparison in Table III, they quantify the resource trade-off of the complete framework.

Figure 5 reports the corresponding trajectories against cumulative modeled latency under the same analytical client profiles. The matched-target results in Table III summarize the modeled-time savings; these values are distinct from end-to-end measured runtime.

We additionally compare the resources required to reach the peak classification accuracy. Table III reports the cumulative-cost and modeled-time savings ($\Delta\overline{C}$ and ΔT) of FedMATRIX-LoRA relative to the least-resource baseline reaching the same target. Across all 11 classification scenarios, FedMATRIX-LoRA uses fewer resources, with median savings of 41.3% in cumulative budget and 42.3% in modeled elapsed time, demonstrating consistently higher resource efficiency across diverse workloads.

C. Scoring Reliability

We examine whether profiled configuration utility is associated with short-term local improvement. At each selected client and checkpoint, candidate configurations share the same rank, matrix-equivalent budget, and training length and include both score-guided and randomly generated feasible subsets. Each configuration is initialized from the same checkpoint, and improvement is measured as its own pre-training validation loss minus its post-training validation loss on held-out local examples. Spearman correlation is computed separately for

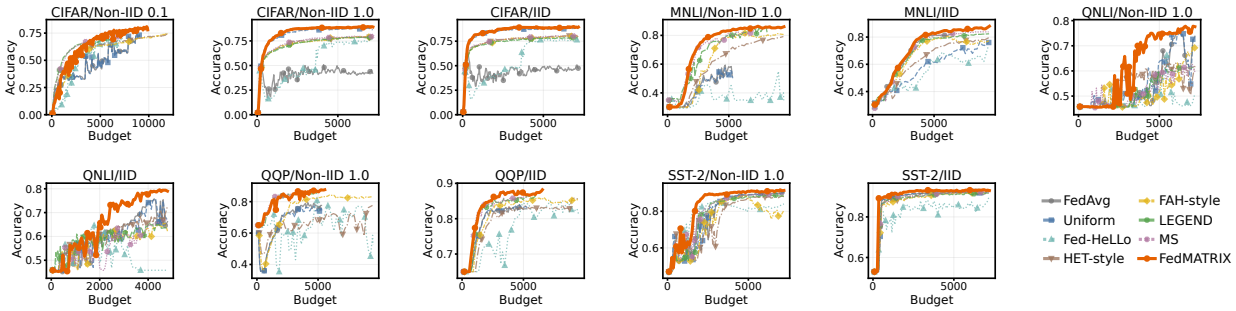


Fig. 4. Budget–quality trajectories for 11 classification scenarios; FedMATRIX-LoRA is emphasized.

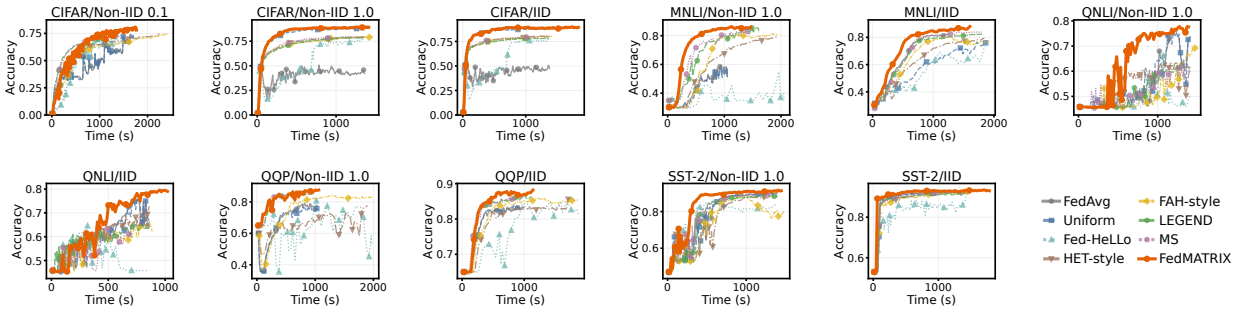


Fig. 5. Modeled-time–quality trajectories for all 11 classification scenarios under the same analytical client profiles as Fig. 4.

TABLE III
RESOURCES REQUIRED TO MATCH EACH STRONGEST BASELINE'S PEAK ACCURACY; POSITIVE SAVINGS FAVOR FEDMATRIX-LORA.

Scenario	Ref.	Target	$\Delta\overline{C}$ (%)	ΔT (%)
CIFAR / Non-IID 0.1	FAH-style	.746	+53.9	+59.4
CIFAR / Non-IID 1.0	Uniform	.882	+42.9	+42.4
CIFAR / IID	Uniform	.886	+54.4	+48.3
MNLI / Non-IID 1.0	LEGEND	.854	+5.6	+27.7
MNLI / IID	MS	.840	+33.2	+42.3
QNLI / Non-IID 1.0	FedAvg	.762	+0.7	+3.2
QNLI / IID	FedAvg	.758	+17.4	+3.2
QQP / Non-IID 1.0	LEGEND	.862	+14.9	+21.7
QQP / IID	LEGEND	.868	+41.3	+44.2
SST-2 / Non-IID 1.0	FedAvg	.902	+44.4	+45.2
SST-2 / IID	HET-style	.930	+47.8	+38.5

each client–checkpoint pair before aggregation. Raw and depth-calibrated scores are evaluated against the same configuration outcomes. **[AUTHOR INPUT: Insert verified A protocol: partition, checkpoint rounds, clients, configurations, steps, validation split, and seeds.] [AUTHOR INPUT: Insert A correlation table/figure and numeric findings; report the aggregation unit and uncertainty.]**

D. Ablation of Client-Aware Matrix Selection

We isolate the scoring source, selection granularity, and depth prior using controlled variants of the same training pipeline. Server-Proxy replaces client-specific scores with scores computed on a separate server-held proxy set. Layer-Level retains local assessment but ties together the candidate matrices within each Transformer block. No-Depth removes only the depth calibration. The full method serves as the reference. All variants share the predefined data splits, candidate-client protocol, optimizer settings, rank, resource limits, and training horizon; the selected participants and matrix subsets may differ as outcomes of the allocation procedure. **[AUTHOR INPUT: Insert verified B metadata and mean/std results for final and best Accuracy, plus actual consumed matrix-equivalent budgets.]**

Relative to Server-Proxy, Layer-Level, and No-Depth, the full method changes final Accuracy by **[AUTHOR INPUT: three signed percentage-point differences]**, respectively, with the corresponding run-to-run variation reported in Table **[AUTHOR INPUT: label]**. The actual consumed budgets are **[AUTHOR INPUT: values]**. These comparisons separate the effects of scoring source, matrix-level choice, and calibration within the evaluated setting.

VI. CONCLUSION

FedMATRIX-LoRA studies client-aware target-matrix selection for resource-constrained federated fine-tuning. Local profiling connects the current adapter to each client's data, while resource-feasible allocation coordinates the resulting preferences across clients. Matrix-wise aggregation supports the resulting heterogeneous update patterns. The reported experiments demonstrate the performance and resource trade-

offs of the complete framework, and the conditional analysis relates its average stationarity to effective-update bias and matrix coverage.

APPENDIX A
PROOFS FOR MATRIX-WISE AGGREGATION

Proof of Theorem 1. Condition on $\mathcal{H}_k$ and set $a = P^k \nabla F(\theta^k)$ and $\bar{v} = \mathbb{E}[v^k \mid \mathcal{H}_k] = a + b^k$. Because the update is supported on P^k, smoothness gives

$$\mathbb{E}[F(\theta^{k+1}) \mid \mathcal{H}_k] \le F(\theta^k) - \gamma\langle a, \bar{v}\rangle + \frac{\beta\gamma^2}{2}\left(\|\bar{v}\|^2 + \mathbb{E}[\|\xi^k\|^2 \mid \mathcal{H}_k]\right). \tag{28}$$

Use $2\langle a, \bar{v}\rangle = \|a\|^2 + \|\bar{v}\|^2 - \|b^k\|^2$ and $\gamma \le 1/\beta$ to discard the nonpositive term $-\gamma(1-\beta\gamma)\|\bar{v}\|^2/2$. Taking total expectations yields

$$\mathbb{E}F(\theta^{k+1}) \le \mathbb{E}F(\theta^k) - \frac{\gamma}{2}\mathbb{E}\|P^k \nabla F(\theta^k)\|^2 + \frac{\gamma}{2}B_{\mathrm{upd}}^2 + \frac{\beta\gamma^2}{2}S_{\mathrm{upd}}^2. \tag{29}$$

Summing over k telescopes the loss terms. The lower bound on F gives

$$\frac{1}{K}\sum_k \mathbb{E}\|P^k \nabla F(\theta^k)\|^2 \le \frac{2D_0}{\gamma K} + B_{\mathrm{upd}}^2 + \beta\gamma S_{\mathrm{upd}}^2. \tag{30}$$

Apply the coverage condition and divide by α_{cov}. □

Proof of Lemma 1. Let $d_i = g_{i,j} - g_j$ and $w_i = \mathbf{1}\{i \in \mathcal{U}_j\}/n_j$. Since $\sum_i d_i = 0$, the selection bias equals $\sum_i (w_i - 1/N) d_i$. Cauchy–Schwarz gives

$$\left\|\sum_i (w_i - 1/N) d_i\right\|^2 \le \sum_i (w_i - 1/N)^2 \sum_i \|d_i\|^2 \tag{31}$$

$$= (1/n_j - 1/N) N \zeta_j^2, \tag{32}$$

which is the claimed bound. For the full effective bias, define the remaining local-update discrepancy as $r = b - s$, with zero selection-bias blocks when no client updates a block. Then $\|b\|^2 \le 2\|s\|^2 + 2\|r\|^2$. The latter term includes local drift, optimizer effects, and any local-model mismatch; the selection lemma alone does not bound it. □

REFERENCES

[1] S. Zhang *et al.*, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[2] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[4] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, nos. 1–2, pp. 1–210, 2021, doi: 10.1561/2200000083.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[6] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 12903–12913, doi: 10.18653/v1/2024.emnlp-main.717.

[7] Z. Gao, Z. Zhang, Y. Guo, and Y. Gong, "Federated adaptive fine-tuning of large language models with heterogeneous quantization and LoRA," in *Proceedings of IEEE Conference on Computer Communications (INFOCOM)*, 2025, pp. 1–10, doi: 10.1109/INFOCOM55648.2025.11044641.

[8] Z. Zhang, P. Liu, J. Xu, and R. Hu, "Fed-HeLLo: Efficient federated foundation model fine-tuning with heterogeneous LoRA allocation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 17556–17569, 2025, doi: 10.1109/TNNLS.2025.3580495.

[9] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10713–10722, doi: 10.1109/CVPR46437.2021.01057.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10088–10115.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97, 2019, pp. 2790–2799.

[12] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 4582–4597, doi: 10.18653/v1/2021.acl-long.353.

[13] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, and Z. Xu, "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 9963–9977, doi: 10.18653/v1/2023.findings-acl.632.

[14] J. Chen, W. Xu, S. Guo, J. Wang, J. Zhang, and H. Wang, "FedTune: A deep dive into efficient federated fine-tuning with pre-trained transformers," *arXiv preprint arXiv:2211.08025*, 2022.

[15] T. Che, J. Liu, Y. Zhou, J. Ren, J. Zhou, V. Sheng, H. Dai, and D. Dou, "Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 7871–7888, doi: 10.18653/v1/2023.emnlp-main.488.

[16] R. Singhal, K. Ponkshe, and P. Vepakomma, "FedEx-LoRA: Exact aggregation for federated and efficient fine-tuning of large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 1316–1336, doi: 10.18653/v1/2025.acl-long.67.

[17] J. Bian, L. Wang, L. Zhang, and J. Xu, "LoRA-FAIR: Federated LoRA fine-tuning with aggregation and initialization refinement," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 3737–3746.

[18] Z. Li, B. Xu, X. Shu, J. Zhang, Y. Yao, G.-S. Xie, and J. Tang, "Tensor-aggregated LoRA in federated fine-tuning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 1058–1067.

[19] H. Wang, X. Wang, Y. Li, J. Chen, J. Zhang, N. Yan, K. He, and W. Wang, "Federated LoRA fine-tuning with pipelined error-mitigated aggregation and matrix-wise freezing," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 5749–5762, doi: 10.18653/v1/2026.findings-acl.284.

[20] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 14457–14483, doi: 10.52202/079017-0461.

[21] J. Koo, M. Jang, and J. Ok, "Towards robust and efficient federated low-rank adaptation with heterogeneous clients," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 416–429, doi: 10.18653/v1/2025.acl-long.19.

[22] X. Huang, Y. Hu, Y.-J. Gong, and X. Zhang, "GMFL: Efficient global masking for federated LLM fine-tuning," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 25293–25312, doi: 10.18653/v1/2026.acl-long.1160.

[23] H. Ban and K. Ji, "Rethinking parameter sharing for LLM fine-tuning with multiple LoRAs," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 12843–12863, doi: 10.18653/v1/2026.findings-acl.625.

[24] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[25] S. Alam, L. Liu, M. Yan, and M. Zhang, "FedRolex: Model-heterogeneous federated learning with rolling sub-model extraction," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 29677–29690.

[26] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 19–35.

[27] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, "FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients," in *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, 2022, pp. 436–449, doi: 10.1145/3498361.3538917.

[28] C. Li, X. Zeng, M. Zhang, and Z. Cao, "PyramidFL: A fine-grained client selection framework for efficient federated learning," in *Proceedings of the 28th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2022, pp. 158–171, doi: 10.1145/3495243.3517017.

[29] Y. G. Kim and C.-J. Wu, "AutoFL: Enabling heterogeneity-aware energy efficient federated learning," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 183–198, doi: 10.1145/3466752.3480129.

[30] B. Luo, X. Li, S. Wang, J. Huang, and L. Tassiulas, "Cost-effective federated learning in mobile edge networks," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 12, pp. 3606–3621, 2021, doi: 10.1109/JSAC.2021.3118436.

[31] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences of the United States of America*, vol. 114, no. 13, pp. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.

[32] J. Liu, Y. Liao, H. Xu, Y. Xu, J. Liu, and C. Qian, "Adaptive parameter-efficient federated fine-tuning on heterogeneous devices," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 12533–12549, 2025, doi: 10.1109/TMC.2025.3586644.

[33] A. Krizhevsky, "Learning multiple layers of features from tiny images," Department of Computer Science, University of Toronto, Toronto, ON, Canada, Technical Report, 2009.

[34] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[35] B. Gliwa, I. Mochol, M. Biesek, and A. Wawer, "SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization," in *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, 2019, pp. 70–79, doi: 10.18653/v1/D19-5409.

[36] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[37] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[38] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao, "AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning," *arXiv preprint arXiv:2303.10512*, version 2, 2023.
[39] Y.-L. Sung, V. Nair, and C. Raffel, "Training neural networks with fixed sparse masks," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.
[40] Z. Zhang, R. Hu, P. Liu, and J. Xu, "Fed-pilot: Optimizing LoRA allocation for efficient federated fine-tuning with heterogeneous clients," *arXiv preprint arXiv:2410.10200*, version 2, 2025.